

FlowType：基于大模型的情景感知文本输入助手

杜明远

2023012155

摘要

键盘文本输入是现代人机交互最频繁、最基础的行为之一。现有商业输入法已经能够依靠统计模型、词频记忆和云端词库完成大多数常规输入，但在桌面任务不断变化的场景中，仍难以及时理解用户当前正在阅读、学习或工作的具体语境。本文总结了 FlowType 的设计与实现：系统不替代用户已经熟悉的商业输入法，而是在用户正常使用输入法组字时，通过桌面 OCR、上下文关键词提取、本地模糊匹配和大模型查询扩展，为输入法候选提供轻量的情景感知增强。面对输入交互高实时性与大模型响应延迟之间的矛盾，FlowType 采用本地词库快速召回与异步模型纠错并行的双路径设计；面对新输入逻辑与用户既有习惯之间的矛盾，系统保持传统候选窗式交互，通过停顿延迟或快捷键触发跟随鼠标的小窗口。实验性开发结果表明，该设计能够在专业术语、长尾作品名和英文模糊输入等场景中补足传统输入法缺失的上下文信息，为个性化、任务感知的输入辅助提供了一种可行路径。

关键词：上下文感知；输入法增强；OCR；模糊匹配；大语言模型

1 项目背景

键盘文本输入是桌面计算环境中最稳定、也最频繁的交互形式。无论是撰写文档、检索资料、整理笔记，还是在即时通讯和浏览器中完成日常交流，用户都需要不断把脑中的意图转化为字符序列。中文输入法承担了从拼音到汉字的解码工作，并在长期发展中形成了高度成熟的交互范式：用户输入拼音，输入法给出候选，用户通过数字键、空格键或方向键选择上屏。由于这一范式已经深度嵌入用户习惯，任何新的输入辅助系统都必须认真面对现有输入法交互的惯性。

目前主流商业输入法已经能够处理大量高频表达，也能够通过用户词库和云端词频改善常见输入体验。然而，它们仍然存在一个结构性痛点：输入法通常只能看到用户正在输入的拼音串和候选选择历史，却难以理解用户当前屏幕上的任务情境。例如，用户正在学习操作系统课件时，屏幕中频繁出现“栈帧”“中断”“页表”等专业术语，但传统输入法并不知道用户此刻处于这一学习场景；用户正在浏览个人游戏库时，输入法也难以知道用户想输入的是某个完整游戏名称，而不是一个更常见的短词。换言之，传统输入法可以较好地利用全局语言统计，却缺少对当前桌面上下文的实时感知。

大语言模型为这一问题提供了新的可能。与传统候选排序不同，大模型能够理解一段文本所描述的主题、任务和语义关系。如果系统可以从用户当前桌面中提取可见文本，



图 1: 现有输入法在上下文缺失场景中的交互痛点示意

再让模型概括当前任务并抽取中英文关键词，那么输入法候选就不必只依赖通用词频，而可以被最近的阅读内容、工作材料和个人任务所补充。进一步地，当用户输入一个不完整或存在错误的拼音串时，模型还可以结合当前任务摘要和商业输入法已经给出的候选提示，推断出更符合语境的表达。这意味着大模型并不一定要替代输入法，而可以成为输入法候选背后的情景感知辅助层。

但是，将大模型能力引入输入过程并不直接等价于良好的输入体验。首先，文本输入具有很强的实时性要求。用户在打字时通常期待候选立即出现，哪怕数百毫秒的额外等待也可能打断思路；而大模型调用往往需要更长时间，且网络链路和模型排队会带来不稳定延迟。因此，如果每次候选生成都依赖模型实时返回，系统很难满足输入交互的节奏。其次，输入法交互是高度习惯化的操作。用户已经熟悉商业输入法的候选窗口、数字选择、空格确认和拼音组字逻辑，若新的系统要求用户改用完全不同的输入方法，学习成本和心理成本都会明显增加。新的输入逻辑不能轻易改变用户已经形成的肌肉记忆。

FlowType 的设计正是围绕这两个矛盾展开。面对实时性矛盾，系统采用“本地快速召回 + 异步模型增强”的双路径：桌面上下文在后台持续提取并写入本地词库，用户触发辅助输入时优先通过本地模糊匹配给出候选，以保证响应速度；同时，系统再异步调用大模型，根据拼音、输入法候选条和最近桌面摘要补充更具语义推理能力的候选。面对交互习惯矛盾，系统并不替换搜狗输入法等商业输入法，而是在用户正常组字的基础上进行增强。当用户对原有候选不满意时，可以通过短暂停留或主动按下快捷键唤出一个跟随鼠标的小窗口，该窗口以类似传统输入法候选条的形式展示候选项，并支持熟悉的键盘确认操作。由此，FlowType 试图在不破坏既有输入习惯的前提下，把大模型对当前任务的理解转化为可用的输入辅助。

2 系统实现

2.1 交互体验设计

FlowType 的核心交互原则是“保持原输入法，增强候选结果”。在开发阶段，系统以 Windows 平台上的搜狗输入法为主要适配对象。用户仍然按照原来的方式输入拼音、观察搜狗输入法的候选条，并在候选合理时直接使用商业输入法完成上屏。只有当候选

不符合当前意图，或用户希望调用上下文辅助时，FlowType 才介入当前输入过程。

系统提供两种触发方式。第一种是主动触发：用户按下 Left Alt 键后，系统读取当前拼音缓冲区和搜狗组字条内容，并弹出辅助候选窗口；当窗口已经打开时，再次按下 Left Alt 可以确认当前高亮候选。第二种是停顿触发：当用户完成一段拼音输入后短暂停留，且系统检测到搜狗候选条仍然可见时，FlowType 会自动判断用户可能正在犹豫，从而弹出辅助窗口。当前实现中的默认停顿阈值为 300ms，这一阈值既能避免每次按键都触发模型逻辑，也能让用户在候选不满意时得到及时反馈。

弹窗的视觉位置和行为也尽量贴近输入法候选窗口。Electron 主进程在收到匹配结果后，根据当前鼠标位置把一个无边框、置顶且不抢焦点的小窗口放置在光标右下方，并在屏幕边界处自动调整位置。窗口每页显示五个候选项，支持 F1-F5 选择当前页候选，也支持方向键翻页和 Enter 确认。候选被确认后，系统先取消搜狗当前组字状态，再通过剪贴板将目标文本插入前台应用，并尽可能恢复用户原有剪贴板内容。这样，FlowType 的交互表现仍然像一个候选条，而不是一个额外的聊天机器人或独立编辑器。

2.2 总体架构

系统由 Electron 前端、本地 Python 子服务和 FastAPI 后端共同组成。Electron 主进程负责窗口管理、进程编排和候选上屏；本地 Python 侧负责屏幕 OCR、全局拼音缓冲、搜狗候选条捕获、关键词匹配和 Windows 自动化；后端则封装面向 Azure OpenAI 的两个模型能力：窗口上下文提取器（Window Context Extractor，简称 WCE）和上下文感知查询扩展器（Context-Aware Query Expander，简称 CAQE）。这些服务默认绑定在本机地址，OCR 服务运行在 127.0.0.1:9000，后端服务运行在 127.0.0.1:8000。

从运行流程看，FlowType 可以分为两条相互配合的链路：后台的上下文情景提取链路，以及前台的辅助输入链路。前者持续观察用户桌面，把可见文本转化为结构化情景词库；后者在用户输入时利用这份词库和模型补全能力，生成候选并完成上屏。两条链路的解耦是系统满足实时性要求的关键，因为耗时较长的屏幕理解过程可以提前在后台完成，而真正发生输入交互时，本地词库已经具备可立即查询的上下文信息。

2.3 上下文情景提取

上下文情景提取链路从屏幕 OCR 开始。系统周期性截取当前桌面图像，并交给本地 PaddleOCR 服务识别可见文字。由于桌面文本并不是线性文档，OCR 原始结果通常存在块顺序混乱、行列关系不清和噪声字符等问题，因此系统会对识别结果进行空间重建：先根据置信度过滤低质量文本，再依据文本框的位置关系进行行分组、列合并和段落整理，最终得到更接近人类阅读顺序的结构化文本。

为了降低模型调用频率，系统不会把每一轮 OCR 结果都发送给后端。当前实现会把本轮重建文本与上一轮成功上送的基线文本计算相似度，只有当相似度低于阈值时，才认为屏幕内容发生了足够明显的变化，并通过 WebSocket 发送给 WCE。这样可以避免用户在同一页面停留时重复消耗模型调用，也让上下文更新更贴近任务切换、页面跳转和文档内容变化等关键时刻。

```
15:33:08 SEND change= 65.7% ocr= 844ms wce= 4037ms lines=28
      theme : CS比赛直播
      summary : 屏幕内容主要是《反恐精英》比赛直播界面，显示地图池包含 Dust2、Mirage、Ancient。
      kw_zh : ['反恐精英', '中路烟', '炼金武库']
      kw_en : ['Dust2', 'Mirage', 'Ancient', 'Team Spirit', 'Team Falcons', 'DANK1NG',
[ocr] INFO: 127.0.0.1:60264 - "POST /ocr HTTP/1.1" 200 OK
15:33:13 skip change= 33.4% ocr= 797ms lines=30 (below threshold)
[ocr] INFO: 127.0.0.1:60265 - "POST /ocr HTTP/1.1" 200 OK
15:33:15 skip change= 36.6% ocr= 859ms lines=28 (below threshold)
```

图 2: OCR 桌面上下文提取过程的运行日志示意

WCE 接收到 OCR 重建文本后，会输出三个层面的结构化信息：当前窗口主题、当前内容摘要，以及中英文关键词列表。主题和摘要用于描述用户正在处理的任务，关键词则被写入本地 SQLite 词库，形成后续输入查询的召回基础。与传统用户词库不同，这份词库并不完全依赖用户历史输入，而是随着用户当前屏幕内容动态更新。例如，用户打开操作系统课件时，词库会倾向于包含“栈帧”“页表”等专业术语；用户浏览游戏库时，词库会倾向于包含完整游戏名和作品名。

2.4 辅助输入与候选生成

前台辅助输入链路从全局键盘缓冲开始。系统通过键盘钩子记录用户最近输入的拉丁字母，并在数字键、空格键或 Enter 等明显完成候选选择的按键出现时清空缓冲。触发辅助输入时，系统不仅使用这段内部拼音缓冲，还会捕获搜狗输入法候选条。实现上，系统枚举搜狗组字窗口，截取候选条图像，并交给本地 OCR 服务识别。由于候选条中通常包含当前拼音前缀和商业输入法已经猜测出的若干候选，FlowType 可以借此校正内部缓冲，并把商业输入法的候选结果作为模型扩展的提示。

候选生成分为本地查询和模型纠错两条路径。第一条路径是本地模糊匹配。系统把 WCE 提取出的中文关键词转换为全拼和首字母索引，也把英文关键词转换为小写形式、CamelCase 子词以及连字符或下划线切分后的子词。查询时，系统使用类似 SymSpell 的删除索引快速召回可能匹配项，再使用面向键盘输入错误的加权编辑距离进行校验和排序。该距离对常见错误给出较低代价：重复字母视为零代价，漏打字母、邻近键误按、相邻字符换位等视为低代价，而不符合输入习惯的大幅替换则直接拒绝。排序时还会综合前缀匹配、最近出现时间、上下文命中次数和词长拟合度。因此，对于已经被屏幕上下文捕获的专有名词，本地路径通常能够在很短时间内给出候选。

第二条路径是 CAQE 模型扩展。该路径把用户拼音串、搜狗候选条 OCR 结果，以及最近一次 WCE 生成的窗口主题和摘要共同发送给后端。模型并不是无条件生成自由文本，而是在“当前语境下，用户这段输入最可能想表达什么”的约束下给出候选。由于模型返回速度相对不稳定，FlowType 把这一路径设计为异步追加：弹窗可以先展示本地结果，随后再把模型候选补充到同一个候选列表中。这样，用户在大多数明确的上下文关键词场景下不需要等待模型；而在长句补全、语义纠错或候选条信息较弱的场景中，又能够利用模型推理能力。

2.5 上屏流程与输入法兼容

候选展示完成后，系统需要把选中的文本插入到用户当前正在编辑的位置。由于 FlowType 不实现一个完整的 TSF 输入法，而是与搜狗输入法协同工作，因此上屏流程必须先处理现有组字状态。当前默认策略是先发送 Esc 取消搜狗输入法正在进行的拼音组字，再把 FlowType 候选通过剪贴板粘贴到前台控件中。为减少对用户环境的影响，系统在粘贴前备份剪贴板内容，完成插入后再尽量恢复。与直接模拟大量键盘输入相比，这种方式更稳定，也更容易在浏览器、文档编辑器和普通桌面应用之间保持一致。

综合来看，FlowType 的实现并不是把大模型放在输入法的每一次按键之后，而是在后台提前提取任务上下文，在前台用本地算法保证响应速度，并把模型能力放在需要语义推断的补充位置。这样的系统结构回应了项目背景中的两个核心矛盾：既避免模型延迟直接阻塞输入，又尽量保留用户熟悉的商业输入法操作路径。

3 交互展示

本节从三个典型场景说明 FlowType 的交互效果。它们分别对应专业学习、生活娱乐和英文术语输入三类高频但传统输入法不容易充分利用上下文的情形。每个场景中，用户都不需要切换输入法或进入额外编辑界面，而是继续使用原有商业输入法；FlowType 只在候选不足时提供贴近输入法候选条的小窗口。

3.1 专业术语：操作系统课件中的“栈帧”

在专业学习场景中，用户可能正在桌面上打开操作系统课程的课件，并一边阅读一边整理笔记。课件中出现了函数调用、内存布局、栈空间和栈帧等概念。对用户而言，“栈帧”是当前任务中非常自然的输入目标；但从通用输入法的角度看，它不是日常高频词，且拼音输入 `zhan'zhen` 时可能更容易被解码为“战争”“战阵”等泛化候选。

FlowType 的处理方式是先在后台通过 OCR 读取课件页面，再由 WCE 提取其中的专业术语并写入本地上下文词库。当用户在笔记软件或浏览器输入框中键入 `zhan'zhen` 并停顿，或主动按下 Left Alt 后，系统会读取当前拼音和搜狗候选条，再对本地词库进行模糊查询。由于“栈帧”刚刚在屏幕上下文中出现，它在词库中具有较高的新鲜度和命中权重，因此可以被快速召回并显示在跟随鼠标的候选窗口中。用户按下对应功能键或确认键后，“栈帧”即可直接上屏。

这一场景体现的是 FlowType 对专业术语的补全能力。系统不是要求用户提前维护一个专业词库，也不是在输入时等待大模型重新理解整页课件，而是把屏幕中已经出现的任务词汇提前转化为本地可查索引。在输入发生时，本地路径即可完成主要召回，从而满足专业场景中“词不常见但刚刚出现过”的输入需求。

3.2 生活娱乐：游戏库中的“饿殍：明末千里行”

生活娱乐场景中的输入目标往往具有强烈的个人上下文。例如，用户正在浏览自己的游戏库，屏幕上出现了作品名“饿殍：明末千里行”。当用户稍后希望在聊天窗口或搜索框中输入这个名称时，商业输入法可能只会根据 e'piao 给出“饿殍”这一短词，而难以自动补全后半部分“明末千里行”。从全局词频看，完整游戏名属于长尾表达；但从当前用户任务看，它恰恰是最相关的候选。

FlowType 在这一场景中的优势来自上下文“见过即可召回”。当用户浏览游戏库时，后台 OCR 已经捕获到界面中的作品名，并通过 WCE 抽取为关键词。用户输入 e'piao 后，系统弹出的候选窗口可以在传统输入法候选之外补充完整表达“饿殍：明末千里行”。如果用户只是想输入前两个字，可以继续使用搜狗候选；如果用户需要完整名称，则可以通过 FlowType 一次性上屏。

这一场景说明，FlowType 的上下文并不局限于严肃办公或专业学习，也适用于用户个人兴趣和日常娱乐内容。商业输入法很难预先知道每个用户的游戏库、书架、音乐列表或最近浏览内容，而 FlowType 可以把这些临时可见的信息转化为短期有效的输入辅助。其价值不在于替代长期用户词库，而在于补足“此刻屏幕上出现过、但全局并不高频”的长尾表达。

3.3 英文模糊输入：正交化算法名称

第三个场景是英文术语的模糊输入。用户在学习线性代数或数值计算内容时，可能刚刚在网页或课件中见过 Gram-Schmidt 正交化算法，但在真正输入时只记得大致拼写，例如输入 gramsch、gram'sch 或不完整的 gram schmit。传统中文输入法对这类英文专有名词的支持通常较弱，尤其是在大小写、连字符、CamelCase 和拼写不完整同时存在时，候选排序很难稳定给出用户想要的结果。

FlowType 的本地词库同时维护中文和英文关键词。对于英文词汇，系统会把原始词转换为小写形式，并根据 CamelCase、下划线和连字符等边界生成子词索引。模糊匹配阶段再结合加权编辑距离处理漏字、相邻字符交换和键盘邻近误按等错误。因此，当上下文中已经出现 Gram-Schmidt 或 GramSchmidt 时，用户只需输入一个近似片段，系统就能够在候选窗口中给出完整词汇“GramSchmidt”或更规范的“Gram-Schmidt”。

这一场景展示了 FlowType 与普通中文输入法不同的另一类能力：它不仅能把拼音映射到中文词，还能把当前上下文中的英文术语纳入同一套输入辅助流程。对于需要频繁输入算法名、函数名、论文术语和软件项目名的用户而言，这种英文模糊召回可以减少复制粘贴和反复回看原文的成本。

除上述三类任务型输入外，FlowType 还可以覆盖带有互联网语境和轻度娱乐色彩的表达。例如“我从丹东来”这一 meme 式句子并不一定是通用输入法的高频候选，但当相关文本已经出现在用户最近浏览内容或聊天上下文中时，系统可以将其作为临时语境线索，用于补充传统候选条无法稳定给出的表达。该例子说明，情景感知输入并不只服务于严肃术语，也可以处理用户在真实社交和网络文化场景中遇到的短期流行表达。



图 3: FlowType 真实交互示例

4 讨论与展望

4.1 隐私与安全边界

输入法位于用户交互链路的底层，任何输入辅助系统都必须严肃处理隐私问题。FlowType 的设计目标不是收集用户输入历史，而是在当前任务中临时利用可见上下文改善候选质量。为此，系统把不同类型的数据尽量限制在本地处理：全屏截图首先交给本机 OCR 服务，搜狗候选条截图也只在本地识别；全局键盘钩子仅维护当前拼音组合所需的短缓冲，不保存完整键盘日志；上下文关键词写入本地 SQLite 词库，用于后续快速匹配。候选上屏过程使用剪贴板作为临时通道，并在完成插入后尽量恢复原剪贴板内容。

与此同时，项目也必须承认大模型能力带来的数据上送边界。当前系统在屏幕内容变化显著时，会把 OCR 重建后的可见文本发送给后端的 WCE，用于提取主题、摘要和关键词；在触发输入辅助时，会把用户拼音、输入法候选条识别结果以及最近的窗口主题和摘要发送给 CAQE，用于生成语义候选。也就是说，系统并不会把每一次按键作为原始日志上传，但会在需要模型理解时上传与当前屏幕语境相关的文本。因此，在产品化阶段，用户必须能够清楚知道哪些数据会离开本机、何时离开本机，以及这些数据会被发送到哪个模型服务。

本项目在开发阶段适配了专用 API 链路，通过后端配置接入 Azure OpenAI，并将本地服务默认限制在 127.0.0.1。这样的实现方式可以避免数据在本机以外的非预期网络路径中流动，也便于后续替换模型服务。面向实际推广时，FlowType 可以进一步允许用户自行提供可靠 API，包括企业内部模型、个人可信代理或本地部署模型。对于隐私敏感用户，还可以提供更细粒度的开关，例如暂停全屏 OCR、限定只对指定应用启用、

对 OCR 文本进行本地脱敏、或仅使用本地词库路径而禁用模型扩展。只有把模型能力和用户可控的数据边界同时呈现出来，输入辅助系统才有可能获得真实使用场景中的信任。

4.2 开发效果总结

从当前开发结果看，FlowType 已经实现了一个围绕商业输入法增强的完整原型。系统能够在后台周期性提取桌面可见文本，使用大模型生成任务摘要和关键词，并把这些关键词转化为本地可查询词库；在前台输入过程中，系统能够捕获用户拼音和搜狗候选条，通过停顿或快捷键弹出跟随鼠标的辅助候选窗口；在候选生成方面，本地模糊匹配提供快速响应，模型扩展提供语义补充，两者共同覆盖专业术语、长尾作品名和英文模糊输入等传统输入法较弱的场景。

更重要的是，该原型验证了一种相对保守但实用的交互路线：不重新发明输入法，也不强迫用户把每次输入都交给大模型，而是在原有输入法不能充分表达用户意图时，提供一个轻量、可忽略、可主动触发的候选增强层。图 3 中的四类真实交互示例分别覆盖了专业术语、长尾作品名、英文术语和网络语境表达，说明系统的上下文增强能力并不局限于单一任务，而是可以服务于学习、娱乐、技术写作和日常社交等多种桌面输入情境。这样的设计使 FlowType 可以利用商业输入法已经成熟的基础能力，同时把系统创新集中在上下文提取、个性化词库和语义纠错上。

4.3 后续工作

后续工作将重点从功能开发转向真实使用评估。首先，需要招募被试开展 in-the-field 实验，让用户在自己的真实桌面环境、真实输入任务和真实应用软件中试用系统。与实验室内预设句子的测试相比，现场实验更能反映输入辅助系统的关键问题，例如停顿触发是否会误打扰用户、候选窗口位置是否遮挡阅读内容、本地匹配结果是否足够稳定、模型候选是否真正减少了用户输入负担，以及用户是否愿意长期保留这样的辅助层。

其次，需要围绕用户体验收集可量化指标和主观反馈。可量化指标包括触发次数、候选点击或确认率、从触发到上屏的延迟、误触发率、用户回退或撤销次数等；主观反馈则包括用户对候选准确性、打断感、隐私可接受性和学习成本的评价。这些数据可以用于继续调整停顿阈值、候选排序权重、上下文更新频率和模型调用策略。

最后，FlowType 还可以继续扩展对不同输入法和不同应用场景的适配。目前开发阶段以搜狗输入法为例，后续可以逐步支持更多商业输入法候选条识别和上屏策略；在算法层面，可以进一步融合用户显式接受记录，让系统逐渐区分“屏幕上出现过”与“用户真正常用”的词；在隐私层面，则可以探索完全本地模型或用户自托管模型链路。总体而言，FlowType 当前已经证明上下文感知输入辅助具有可行性，下一阶段的关键是通过真实用户反馈，把这一能力打磨成稳定、可信且不会打断输入流的产品体验。